



SafeSpace AI

RAG-Based Mental Health Support Chatbot

NLP Final Project

Team Members

Mohamed Emad · Ziad Mahmoud

GitHub Repositories

The two repositories that make up the full SafeSpace AI system are listed below. They are the primary deliverable alongside the four module notebooks.

 **AI + FastAPI Backend** <https://github.com/3omdawyl/SafeSpaceAI>

NLP pipeline · RAG engine · REST API (FastAPI) · All four module implementations

 **React Frontend**

<https://github.com/ZeyadMahmoudAmrMohamed/SafeSpaceAI>

Conversational UI · Real-time chat · Connects to the FastAPI backend

Project Overview

SafeSpace AI is an end-to-end, retrieval-augmented generation chatbot designed to provide grounded, empathetic, and context-aware support for users dealing with anxiety, depression, stress, and related topics. The system is built as an NLP course capstone that integrates four distinct modules into a single coherent pipeline.

The pipeline is fully conversational: a user message flows through language detection, emotion classification, and intent routing before reaching the RAG engine, which retrieves relevant mental health knowledge and generates a final response — all in the user's original language.

System Architecture

The pipeline is composed of four sequential NLP modules. Each module's output feeds directly into the next, making the system tightly integrated rather than a collection of independent tasks.

#	Module	Description
1	Language Detection	Identifies the user's language (20 languages via TF-IDF + ML). Non-English input is translated before processing.
2	Emotion Classification	Fine-tuned BiLSTM classifies the emotional tone of the query, enabling empathetic response shaping.
3	Intent Classification	Zero-shot / few-shot LLM prompting routes the query: greeting → direct reply; mental health → RAG pipeline.
4	RAG Pipeline	Hybrid BM25 + semantic search over Qdrant retrieves relevant knowledge. Groq LLM generates the final grounded response.
↩	Translation Back	If the input was non-English, the response is translated back to the original language before delivery.

Module Details

Module 1: Language Detection *[TF-IDF · Logistic Regression]*

A traditional NLP multi-class classifier that identifies the language of an incoming user query across 20 languages. This module is foundational: it gates the translation step and ensures all downstream modules work on English text.

- Dataset: Language Identification Dataset (90k samples from HuggingFace)
- Vectorization: TF-IDF Vectorizer on character n-grams
- Classifier: Logistic Regression / traditional ML
- Output: Detected language code (e.g., 'en', 'ar') + confidence score
- Non-English queries are translated to English before continuing

Module 2: Emotion Classifier [BiLSTM + Synthetic data generation]

A deep learning multi-class classifier that identifies the emotional state expressed in the user's query. The detected emotion directly shapes how the RAG module frames its final response, enabling empathetic and tone-appropriate replies.

- Dataset: Emotion Dataset (6k Twitter messages, HuggingFace) + we synthesized a total of 16,112 entries using llama-3.3-70b-versatile to handle the class imbalance.

'joy' (Original: 5286, Synthetic: 0).
'sadness' (Original: 4512, Synthetic: 774).
'anger' (Original: 2108, Synthetic: 3178).
'fear' (Original: 1850, Synthetic: 3436).
'love' (Original: 1291, Synthetic: 3995),
'surprise' (Original: 557, Synthetic: 4729)

- Model: BiLSTM fine-tuned on Kaggle GPU
- Emotion classes: joy, sadness, anger, fear, love, surprise
- Output: Emotion label

Module 3: Intent Classifier [Zero-shot / Few-shot LLM Prompting (Groq)]

Classifies the user's intent into one of five categories, routing the system to the correct response path without any additional model training. This avoids unnecessary RAG calls for simple conversational turns.

- Method: few-shot prompting via Groq LLM
- Intent classes: greeting · goodbye · gratitude · asking_mental_health_question · out_of_scope
- greeting / goodbye / gratitude → direct response, no RAG
- asking_mental_health_question → triggers the full RAG pipeline
- out_of_scope → polite decline with redirection

Module 4: Q&A RAG Pipeline [Qdrant · Sentence Transformers · Groq LLM]

The core knowledge-retrieval and generation module. When a query is classified as a mental health question, this pipeline retrieves the most relevant passages from the knowledge base and synthesizes a grounded, empathetic response.

- Dataset: Mental Health Counseling Conversations (17k professional Q&A pairs, HuggingFace) + curated PDFs we’ve collected from the domain
- Embeddings: all-MiniLM-L6-v2 (Sentence Transformers) — compact and fast
- Vector DB: Qdrant Cloud (HNSW indexing)
- Search: Hybrid — BM25 keyword search + semantic similarity
- Query expansion: HyDE (Hypothetical Document Embeddings) for better retrieval
- NER extraction: identifies symptoms and triggers from the query
- LLM: Groq free tier (gpt-oss-120b or gpt-oss-20b) for final response generation
- Response is conditioned on retrieved chunks + detected emotion

Technology Stack

Python 3.13+ — Core language for all modules and scripts	FastAPI — REST API server with interactive /docs
React — Frontend conversational UI	PyTorch — Deep learning backend for BiLSTM
Hugging Face — Transformers, datasets, and model hub	Sentence Transformers — all-MiniLM-L6-v2 embeddings
Qdrant — Cloud vector database with HNSW indexing	Groq — Free-tier LLM inference API
scikit-learn — TF-IDF, Logistic Regression, preprocessing	Weights & Biases — Experiment tracking during training
pandas / numpy — Data manipulation and analysis	uvicorn — ASGI server for FastAPI

Setup & Usage

Environment Variables

Create a .env file in the project root with the following keys:

Variable	How to obtain
GROQ_API_KEY	https://console.groq.com/
QDRANT_URL	https://cloud.qdrant.io/
QDRANT_API_KEY	Generated from Qdrant dashboard
WANDB_API_KEY	wandb login locally or set in Kaggle secrets

Running the Pipeline

From the backend repository root, run the steps below in order:

- `pip install -r requirements.txt`
- `python scripts/00_prepare_data.py` → downloads datasets, chunks PDFs
- `python scripts/01_train_language_detector.py` → trains TF-IDF classifier
- `python scripts/03_setup_rag.py` → indexes knowledge base to Qdrant
- `uvicorn app.main:app --reload` → starts the FastAPI server on localhost:8000
- `python scripts/04_test_pipeline.py` → runs full pipeline smoke tests

API Reference

POST /chat

Main inference endpoint. Accepts a user message and returns a full pipeline response.

Response Field	Description
response	Generated text response (in original language if translated)
emotion	Detected emotion label (e.g., fear, sadness)

language	Detected language code (e.g., en, ar)
intent	Classified intent (e.g., asking_mental_health_question)
confidence_scores	Per-module confidence values (language, emotion, intent)
sources	Retrieved knowledge chunks with source attribution

Datasets

Dataset	Size	Classes/Type	Source
Language Identification	~90,000 samples	20 languages	HuggingFace — auto-downloaded
Emotion Detection	~6,000 samples	6 emotion classes	HuggingFace (Twitter messages)
Mental Health Counseling	~17,000 Q&A pairs	Professional counseling	HuggingFace — core RAG knowledge
Mental Health PDFs	Custom	Curated books/guides	Manually placed in data/raw/

Limitations

- Not a therapist — responses are AI-generated from training data, not from licensed mental health professionals. Always direct users in genuine distress to real resources.
- Knowledge base quality — RAG output quality is directly bounded by the quality and coverage of the indexed PDFs and counseling dataset.
- English-biased training — the emotion classifier and LLM are trained/prompted primarily in English. Translation quality for low-resource languages may vary.
- No persistent user state — this is a proof-of-concept; there is no cross-session conversation history or user profile.
- API rate limits — the free Groq tier (~30 requests/min) is sufficient for development but not for production workloads.

Future Work

- Expand and improve the knowledge base with additional peer-reviewed mental health resources and clinical guidelines.
- Fine-tune the LLM on domain-specific mental health data for more nuanced and accurate responses.
- Implement persistent conversation history and user context across sessions.
- Extend multi-language support end-to-end, including multilingual emotion classification.
- Integrate a feedback-driven fine-tuning loop using collected user ratings.

Team

<u>Mohamed Emad</u>	<u>Ziad Mahmoud</u>
<u>Frontend Repo</u>	<u>Backend Repo</u>